<<route>>
DisputeUserRoutes
+GET /disputes (AuthGuard, PermissionGuard)
+POST /disputes (AuthGuard, PermissionGuard)
+POST /disputes/upload (AuthGuard, FileInterceptor)
+POST /disputes/:id/evidence (AuthGuard, PermissionGuard)
+POST /disputes/:id/counter-offer (AuthGuard, PermissionGuard)
+POST /disputes/:id/accept-offer (AuthGuard, PermissionGuard)
+POST /disputes/:id/escalate (AuthGuard, PermissionGuard)
+POST /disputes/:id/vote-decision (AuthGuard, PermissionGuard)
+GET /disputes/milestone/:milestoneId (AuthGuard, PermissionGuard)
+GET /disputes/:id (AuthGuard, PermissionGuard)
<<route>>
DisputeExpertRoutes
+GET /expert/disputes (AuthGuard, PermissionGuard)
+GET /expert/disputes/:id (AuthGuard, PermissionGuard)
+POST /expert/disputes/:id/claim (AuthGuard, PermissionGuard)
+POST /expert/disputes/:id/decision (AuthGuard, PermissionGuard)
maps to
maps to
calls
DisputeUserController
-disputeUserService: DisputeUserService
+listMyDisputes(userId, query)
+createDispute(userId, body)
+uploadFile(file)
+addEvidence(userId, disputeId, body)
+submitCounterOffer(userId, disputeId, body)
+acceptOffer(userId, disputeId)
+escalateToExpert(userId, disputeId)
+voteExpertDecision(userId, disputeId, body)
+getDisputeByMilestoneId(userId, milestoneId)
+getDisputeDetail(userId, disputeId)
DisputeExpertController
-disputeExpertService: DisputeExpertService
+listDisputes(expertUserId, roleName, query)
+getDisputeDetail(expertUserId, roleName, disputeId)
+claimDispute(expertUserId, roleName, disputeId)
+expertDecision(expertUserId, roleName, disputeId, body)
AuthGuard
-tokenService: TokenService
-prisma: PrismaService
-reflector: Reflector
+canActivate(context: ExecutionContext): Promise<boolean>
-extractTokenFromHeader(request: Request): string | undefined
PermissionGuard
-reflector: Reflector
-sharedPermissionRepository: SharedPermissionRepository
+canActivate(context: ExecutionContext): Promise<boolean>
<<dto>>
CreateDisputeBodyDTO
+milestoneId: number
+reason: string
+description: string
+freelancerAmount?: number
+clientAmount?: number
+files?: DisputeFileItemType[]
<<dto>>
CounterOfferBodyDTO
+freelancerAmount: number
+clientAmount: number
+reason?: string
+files?: DisputeFileItemType[]
<<dto>>
ExpertDecisionBodyDTO
+freelancerAmount: number
+clientAmount: number
+reason: string
calls
calls
<<dto>>
VoteExpertDecisionBodyDTO
+agree: boolean
<<type>>
DisputeDetailType
+id: number
+milestoneId: number
+openedById: number
+respondentId: number
+status: DisputeStatusEnum
+negotiationDeadline: Date
+freelancerAmount: number | null
+clientAmount: number | null
+expertId: number | null
+expertFreelancerAmt: number | null
+expertClientAmt: number | null
+clientAgreed: boolean | null
+freelancerAgreed: boolean | null
<<type>>
DisputeEvidenceTypeModel
+id: number
+disputeId: number
+submittedById: number
+type: DisputeEvidenceTypeEnum
+comment: string | null
DisputeUserService
-disputeUserRepo: DisputeUserRepository
-paymentService: PaymentService
-cloudinaryService: CloudinaryService
-logger: Logger
+uploadFile(file: Express.Multer.File)
+listMyDisputes(userId, query)
+getDisputeDetail(userId, disputeId)
+getDisputeByMilestoneId(userId, milestoneId)
+createDispute(userId, body)
+addEvidence(userId, disputeId, body)
+submitCounterOffer(userId, disputeId, body)
+acceptOffer(userId, disputeId)
+escalateToExpert(userId, disputeId)
+voteExpertDecision(userId, disputeId, body)
DisputeExpertService
-disputeExpertRepo: DisputeExpertRepository
-logger: Logger
-getActiveExpertProfile(expertUserId)
+listDisputes(expertUserId, roleName, query)
+getDisputeDetail(expertUserId, roleName, disputeId)
+claimDispute(expertUserId, roleName, disputeId)
+expertDecision(expertUserId, roleName, disputeId, body)
DisputeUserRepository
-prisma: PrismaService
+findMilestoneWithContract(milestoneId: number)
+findDisputeById(disputeId: number)
+findDisputeByMilestoneId(milestoneId: number)
+listDisputes(query, userId)
+createDisputeTransaction(data)
+addEvidenceTransaction(data)
+submitCounterOfferTransaction(data)
+escalateToExpert(disputeId: number)
+voteExpertDecisionTransaction(data)
+finalizeDisputeTransaction(data)
calls
DisputeExpertRepository
-prisma: PrismaService
+findDisputeById(disputeId: number)
+findUserWithProfileAndExpert(userId: number)
+listDisputesForExpert(query, expertExpertise, expertId)
+claimDisputeAtomic(disputeId, expertId)
+expertDecisionTransaction(data)
maps to
maps to
maps to
maps to
<<schema>>
Dispute
+id: number [PK]
+milestoneId: number [FK, Unique]
+openedById: number [FK]
+respondentId: number [FK]
+reason: string
+description: string
+status: DisputeStatus
+negotiationDeadline: Date
+freelancerAmount: Decimal | null
+clientAmount: Decimal | null
+lastOfferById: number | null
+expertId: number | null [FK]
+expertReason: string | null
+expertFreelancerAmt: Decimal | null
+expertClientAmt: Decimal | null
+clientAgreed: boolean | null
+freelancerAgreed: boolean | null
+resolvedAt: Date | null
1..*
<<schema>>
DisputeEvidence
+id: number [PK]
+disputeId: number [FK]
+submittedById: number [FK]
+type: DisputeEvidenceType
+comment: string | null
+createdAt: Date
0..*
<<schema>>
DisputeFile
+id: number [PK]
+disputeId: number [FK]
+evidenceId: number | null [FK]
+uploadedById: number [FK]
+fileName: string
+fileUrl: string
+fileSize: number | null
+fileType: string | null
0..1
<<schema>>
Milestone
+id: number [PK]
+contractId: number [FK]
+title: string
+amount: Decimal
+status: MilestoneStatus
+paymentStatus: MilestonePaymentStatus
1..*
<<schema>>
Contract
+id: number [PK]
+clientId: number [FK]
+freelancerId: number [FK]
+status: ContractStatus